



PandaBuilder

Tutorial

The screenshot shows the PandaBuilder application window with a menu bar (File) and several configuration panels:

- CPU Options:** Includes a 'Generics' dropdown set to 'debug_on', a text field with 'false', and a 'Change' button.
- Available Components:** Includes a dropdown set to 'input port' and an 'Add' button.
- Component Options:** This panel is divided into two columns.
 - Left column: 'Component Selection' dropdown (set to 'input port'), a 'Remove' button, 'Generics' dropdown (set to 'port_width'), a text field with '8', a 'Change' button, and an 'External Component' checkbox (unchecked).
 - Right column: 'Name' text field (set to 'input port'), 'ID' text field (set to 'pin'), 'Base Address (Hex)' text field (set to '110'), 'Address Bits' text field (set to '0'), 'Input Data' checkbox (checked), 'Interrupt' checkbox (unchecked), 'Interrupt Priority' text field (set to '0'), and a 'Change' button.
- System:** Includes a 'System Name' text field (set to 'NewSystem'), a 'Pack ROM into System' checkbox (unchecked), 'DataMux Style' radio buttons (selected: 'Asynchronous'), 'ROM Type' radio buttons (selected: 'Delayed clk'), 'Instruction Address Width (1..10)' text field (set to '10'), and 'Data Address Width (1..10)' text field (set to '10').

At the bottom, there are four buttons: 'Change', 'Save', 'Load', 'Generate', and 'Create SW Files'.

Logic Solutions Bründler
Winkelbühlrain 10
CH-6043 Adligenswil
Switzerland
Tel. +41 (0)76 408 83 58
E-Mail info@logicsolutions.ch
Internet www.logicsolutions.ch

Table of Contents

1 Legal Notices.....	3
2 System overview	4
3 System Creation.....	5
3.1 System Settings.....	5
3.2 CPU Options.....	5
3.3 Add RAM.....	6
3.4 Add 4-Bit Input Port.....	7
3.5 Add 4-Bit Output Port.....	8
3.6 Add 8-Bit Timer.....	9
3.7 Add External Inverter Component.....	10
3.8 Save System.....	10
3.9 Generate System.....	10
4 Software Creation.....	11
4.1 Write Software using the Template.....	11
4.1.1 Delete Macros.....	11
4.1.2 Set Memory Base.....	11
4.1.3 Register Definitions for External Inverter Component.....	11
4.1.4 Main Program.....	12
4.1.5 Interrupt Service Routines.....	13
4.1.6 Overview.....	14
4.2 Create ROM Files.....	16
5 Running the Design in Hardware.....	17
5.1 Import Files.....	17
5.2 Create Symbol Files.....	17
5.3 Connect External Inverter.....	18
5.4 Compilation Results.....	18

Illustration Index

Illustration 1: System Overview.....	4
Illustration 2: System Settings.....	5
Illustration 3: CPU Options.....	5
Illustration 4: RAM Settings.....	6
Illustration 5: 4-Bit Input Port Settings.....	7
Illustration 6: 4-Bit Output Port Settings.....	8
Illustration 7: 8-Bit Timer Settings.....	9
Illustration 8: External Inverter Component Settings.....	10
Illustration 9: System Directory.....	10
Illustration 10: Macro Definitions.....	11
Illustration 11: Variable Definitions.....	11
Illustration 12: Inverter Register Definitions.....	11
Illustration 13: Main Program.....	12
Illustration 14: Interrupt Service Routines.....	13
Illustration 15: Assemble Sourcefile.....	16
Illustration 16: Create ROM Files.....	16
Illustration 17: Add Files to Project.....	17
Illustration 18: Create Symbol Files.....	17
Illustration 19: Connections of the external Inverter.....	18

Abstract

This document leads the reader step for step through the development of a Panda system using the PandaBuilder software.

In the second part of the document, where the system is compiled and downloaded on a chip, altera tools are used.

1 Legal Notices

All copyrights are reserved by Logic Solutions Bründler. If you want to use this software for commercial issues, you have to buy it.

It is not allowed to distribute the software, any of the VHDL files or parts of it to third-persons.

For any sales or legal questions please write an e-mail to info@logicsolutions.ch

2 System overview

Before you start with the tutorial please install all applications as described in the document *PandaBuilder_UserGuide.pdf*.

The system should read the state of the 4 buttons and write it continuously to 4 LEDs. An external component controls 4 other LEDs. They display 1 second the state of the 4 buttons and then 1 second their inverted state and so on.

Therefore the system contains the following components:

- CPU
- 4 Bit input port
- 4 Bit output port
- 8 Bit timer (resolution 10 ms)
- RAM (16 Byte),
- Inverter component (is implemented as external component)

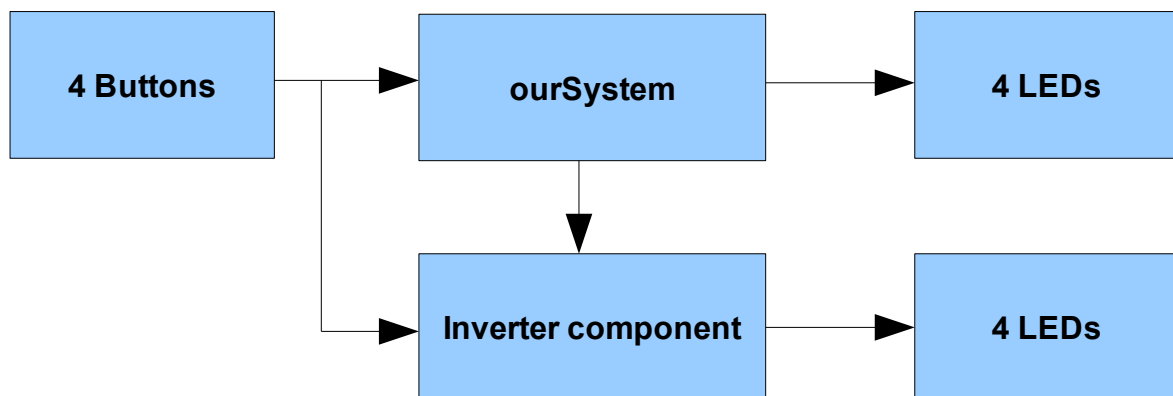


Illustration 1: System Overview

For the Hardwaretest the MaxII Micro Kit from terasIC will be used. It contains 4 Buttons, 8 LEDs and a 50MHz oscillator.

3 System Creation

3.1 System Settings

Set the following system settings and apply them with a click on the „Change“ button.

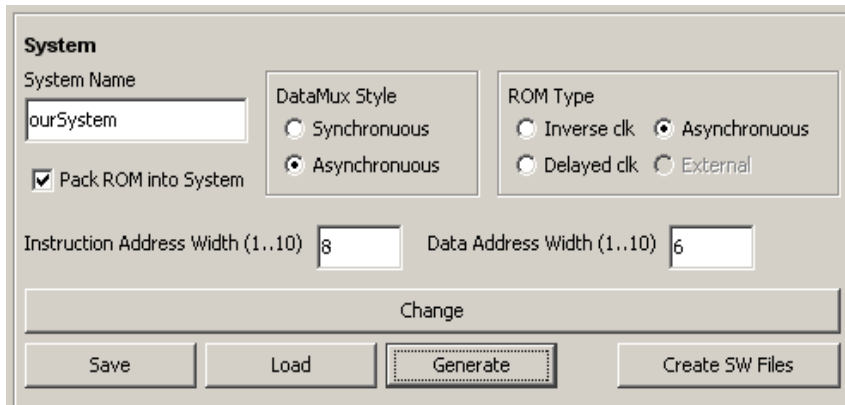


Illustration 2: System Settings

System Name	ourSystem	Name of the System
DataMux Style	Asynchronous	Default
Rom Type	Asynchronous	Default
Instruction Address Width	8	256 Instructions are enough for this task
Data Address Width	6	64 Memory addresse are enough
Pack ROM into System	yes	Reduces the number of ext. connections

3.2 CPU Options

Set the generics to their new values and apply each one with a click on the „Change“ button.

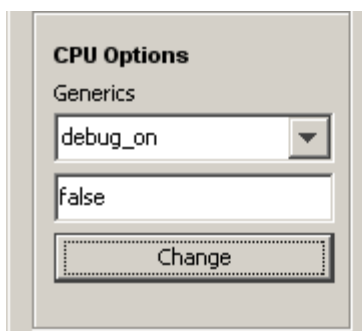


Illustration 3: CPU Options

debug_on	false	No debugger
indir_on	false	No indirect addressing
carry_arith_on	true	Carry arithmetics on
clk_frequency	50000000	50MHz clock
user_stack_on	false	No Stack
stack_depth	2	2 level stack
mac_for_mul	false	No MAC
mul_on	false	No multiplication
num_of_ints	1	One interrupt (timer)
adv_skips_on	true	Advanced skips on
dat_on_sstack	false	No data on system stack
interrupts_on	true	Interrupts are supported

3.3 Add RAM

Select „memory“ from the *Available Components* list and press the „Add“ button. Then change the setting as shown in the image.

Illustration 4: RAM Settings

Generics	address_width	4	16 Bytes of RAM are used
Settings	Adress Bits	4	16 Bytes of RAM are used

3.4 Add 4-Bit Input Port

Select „input port“ from the *Available Components* list and press the „Add“ button. Then change the setting as shown in the image.

The screenshot shows the PandaBuilder configuration window. On the left, under 'Available Components', 'input port' is selected. On the right, under 'Component Options', 'Buttons' is selected in the 'Component Selection' dropdown. The 'Generics' section shows 'port_width' set to 4. The 'Name' field is 'Buttons', the 'ID' is 'but', and the 'Base Address (Hex)' is '10'. The 'Adress Bits' field is '0'. The 'Input Data' checkbox is checked, and the 'Interrupt' checkbox is unchecked. The 'Interrupt Priority' is set to 0.

Illustration 5: 4-Bit Input Port Settings

Generics	port_width	4	4 buttons must be read
	synchronize	true	The buttons must be synchronized

3.5 Add 4-Bit Output Port

Select „output port“ from the *Available Components* list and press the „Add“ button. Then change the settings as shown in the image.

CPU Options

Generics

debug_on

false

Available Components

output port

Component Options

Component Selection: LEDs

Generics

port_width

4

☐ External Component

Name: LEDs

ID: led

Base Address (Hex): 11

Address Bits: 0

☒ Input Data

☐ Interrupt

Interrupt Priority: 0

Illustration 6: 4-Bit Output Port Settings

Generics port_width 4 4 LEDs must be driven

3.6 Add 8-Bit Timer

Select „8bit timer“ from the *Available Components* list and press the „Add“ button. Then change the settings as shown in the image.

The screenshot shows the PandaBuilder configuration window. On the left, under 'CPU Options', 'debug_on' is set to 'false'. Below that, in the 'Available Components' list, '8bit timer' is selected. On the right, the 'Component Options' panel is active for the 'Timer' component. It shows the following settings: Name 'Timer', ID 'tmr', Base Address (Hex) '14', Address Bits '2', 'Input Data' checked, 'Interrupt' checked, and 'Interrupt Priority' set to '1'. There are 'Change' buttons for the Base Address, Address Bits, and the overall component settings.

Illustration 7: 8-Bit Timer Settings

Generics	clk_frequency	50000000	50 MHz clock
	resolutions_us	10000	10ms resolution
Settings	Base Address	0x14	Last two bits must be zero because 2 address bits are used

3.7 Add External Inverter Component

Select „external component“ from the *Available Components* list and press the „Add“ button. Then change the setting as shown in the image.

The screenshot shows the 'Component Options' dialog for an 'Inverter' component. The 'Available Components' list on the left has 'external component' selected. The 'Component Options' section on the right shows the following settings:

- Component Selection:** Inverter
- Name:** Inverter
- ID:** inv
- Base Address (Hex):** 12
- Address Bits:** 0
- Input Data:** ☐ (unchecked)
- Interrupt:** ☐ (unchecked)
- Interrupt Priority:** 0
- External Component:** ☒ (checked)

Buttons for 'Remove', 'Change', and 'Add' are visible in the interface.

Illustration 8: External Inverter Component Settings

Settings	Address Bits	0	The external Inverter has only one register
	Input Data	Off	The external Inverter cannot be read from the CPU
	Interrupt	Off	The external Inverter does not generate interrupts

3.8 Save System

Press the „Save“ button and save the system to a directory on your computer. Note that all Files are generated in subdirectories of this directory.

3.9 Generate System

First press the „Generate“ button. Then press the „Create SW Files“ button.

If you open the directory you created in 3.8 it should look like this:

 ourSystemDocs	<i>ourSystemDocs</i>	Documentation files for all components of the system
 ourSystemHW	<i>ourSystemHW</i>	HW design files of the system (all in VHDL)
 ourSystemSW	<i>ourSystemSW</i>	SW files of the system and HW design files for the ROM
 ourSystem.sdf	<i>ourSystem.sdf</i>	The system is saved in this file

Illustration 9:
System Directory

4 Software Creation

Open the file <Project Root>\outSystemSW\ourSyste.asm with the *Crimson Editor* and enable syntax highlighting for panda.

To view the register definitions you can open the file<Project Root>\outSystemSW\regdef.inc too.

4.1 Write Software using the Template

4.1.1 Delete Macros

We don't need macros in this project. Therefore the macro definitions (see Illustration 10: Macro Definitions) can be deleted.

```
;*****
;*** Macros ***
;*****
[MACRO]      MACRO
              [INSTRUCTIONS]
            ENDMACRO
```

Illustration 10: Macro Definitions

4.1.2 Set Memory Base

Change the variable definitions as follows:

```
;*****
;*** Variables ***
;*****
          DSEG      VARIABLES
          REGORG     RAM_BASE
accu_sav  RMW       1           ;Save accu vor interrupt
last_inv  RMW       1
```

Illustration 11: Variable Definitions

4.1.3 Register Definitions for External Inverter Component

Because the inverter component is a user component, PandaBuilder does not know its registers. Therefore you have to define them byself. This is one be the following codesegment which is inserted between the variable definitions and the main programm.

```
;*****
;*** Registers ***
;*****
          DSEG      INVERTER
          REGORG     INV_BASE
INV_EN    RMW       1           ;Inverter Enable Register
```

Illustration 12: Inverter Register Definitons

4.1.4 Main Program

Change the main program as shown in the following illustration.

```

;*****
;*** Main Programm ***
;*****
                CSEG    CODE
                ORG      000h

init            ;Initialization

                ;Variables
                REG_SET  last_inv,    #0 ;Clear last_inv

                ;LEDs
                REG_SET  LED_POUT,    #0 ;Clear all LEDs

                ;Timer
                TMR_INIT_CONTINT    #100 ;Timer generates an interrupt every second

                ;Interrupts
                SET       IE                ;Enable Interrupts

main            ;Main Loop
                LDA      BUT_PIN            ;Read Buttons and write their state to
                STA      LED_POUT          ;... the LEDs
                JMP      main              ;...

                CSEG    INTERRUPTS
                ORG      0ffh
                JMP      tmr_isr          ;Timer Interrupt Vector

```

Illustration 13: Main Program

4.1.5 Interrupt Service Routines

Change the interrupts service routines as follows.

```
;*****
;*** Interrupt Service Routines ***
;*****
        CSEG  INTERRUPT_SERVICE
;***Timer Interrupt Service Routine ***
tmr_isr  STA   accu_sav           ;Save Accu

        TMR_CLRINT               ;Clear Interrupt
        LDA   last_inv           ;Invert inverter state
        INV
        STA   last_inv           ;..
        STA   INV_EN             ;..

        LDA   accu_sav           ;Restore Accu
        RETI
```

Illustration 14: Interrupt Service Routines

Note that the carry-bit is not saved and restored because it is not changed in this interrupt service routine.

4.1.6 Overview

The file should now look like this:

```
#CPU PANDA ;Tells the Assembler which Assembler-Table to use
;*****
;*** Assembler File for ourSystem ***
;*****

#SYMLIST ;Create Symbol List at the end of the Listing
#include "pandamac.inc" ;Include Pseudo Instructions for Panda
#include "regacc.inc" ;Include Register Access Macros for Panda
#include "regdef.inc" ;Include Register definitions for ourSystem
#include "tmr.drv" ;Include Driver for Timer

;*****
;*** Variables ***
;*****
                DSEG    VARIABLES
                REGORG   RAM_BASE
accu_sav        RMW      1 ;Save accu vor interrupt
last_inv        RMW      1

;*****
;*** Registers ***
;*****
                DSEG    INVERTER
                REGORG   INV_BASE
INV_EN          RMW      1 ;Inverter Enable Register

;*****
;*** Main Programm ***
;*****
                CSEG    CODE
                ORG      000h

init            ;Initialization

                ;Variables
                REG_SET  last_inv,    #0 ;Clear last_inv

                ;LEDs
                REG_SET  LED_POUT,    #0 ;Clear all LEDs

                ;Timer
                TMR_INIT_CONTINT    #100 ;Timer generates an interrupt every second

                ;Interrupts
                SET      IE          ;Enable Interrupts
```

```
main          ;Main Loop
              LDA    BUT_PIN          ;Read Buttons and write their state to
              STA    LED_POUT         ;.. the LEDs
              JMP    main             ;..

              CSEG    INTERRUPTS
              ORG     Offh
              JMP     tmr_isr         ;Timer Interrupt Vector

;*****
;*** Interrupt Service Routines ***
;*****
              CSEG    INTERRUPT_SERVICE
;***Timer Interrupt Service Routine ***
tmr_isr       STA     accu_sav        ;Save accu

              TMR_CLRINT              ;Clear Interrupt
              LDA     last_inv        ;Change inverter state
              INV
              ;..
              STA     last_inv        ;..
              STA     INV_EN          ;..

              LDA     accu_sav        ;Restore accu
              RETI
```

4.2 Create ROM Files

Assemble the file *ourSystem.asm*

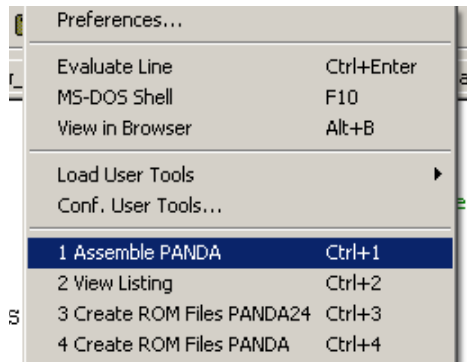


Illustration 15: Assemble Sourcefile

Create ROM-Files because the „Pack ROM into System“ option was enabled when the system was generated.

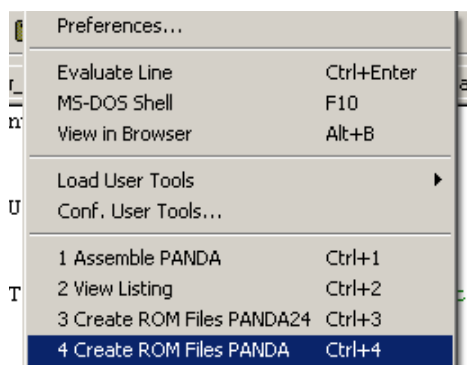


Illustration 16: Create ROM Files

5 Running the Design in Hardware

For this step the Altera software Quartus II and a terasIC MaxII Micro Kit are used.

5.1 Import Files

Copy the design file for the external inverter from the directory *<TutorialFiles>* into the *<Project Root>* directory.

Add the following files over the menue „Project“ > „Add/Remove Files in Project“:

- All files from the directory *<Project Root>\ourSystemHW*
- ROM files from the directory *<ProjectRoot>\ourSystemSW*
AsyncVHDL_ourSyste.vhd
SyncVHDL_ourSyste.vhd
- *<Project Root>\ext_inv.vhd*

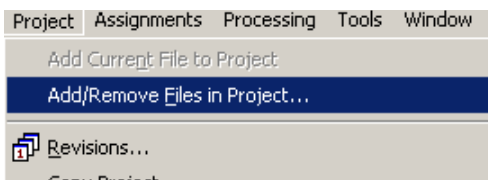


Illustration 17: Add Files to Project

5.2 Create Symbol Files

Create Symbol files for the system file (*ourSystem_system.vhd*) and the external inverter (*ext_inv.vhd*).

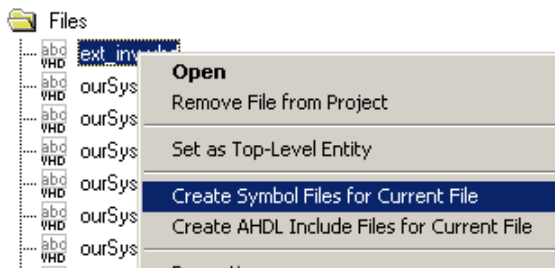


Illustration 18: Create Symbol Files

5.3 Connect External Inverter

Connect the blocks as shown in the illustration:

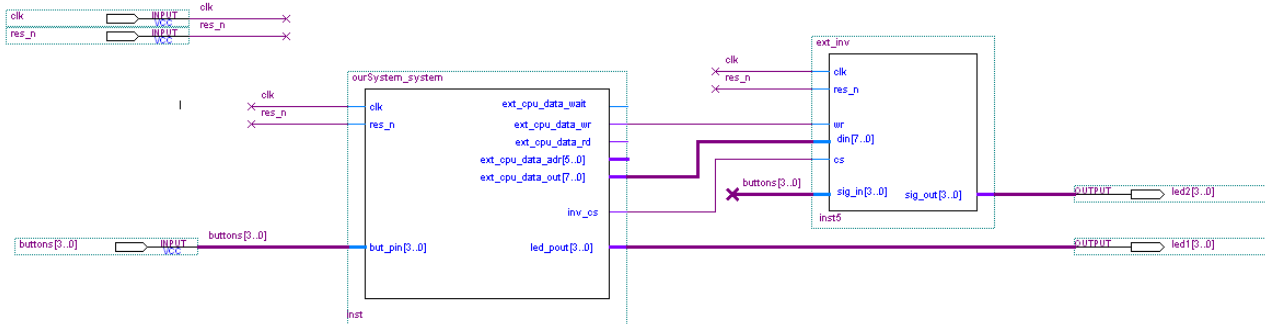


Illustration 19: Connections of the external Inverter

5.4 Compilation Results

Assign pins, compile the design, download and test it.

With Quartus II v8.0 the compilation results should look like this:

- Ressource Usage 534 LE
 - Button Input Port 8 LE
 - Interconnect 4 LE
 - LED Output Port 5 LE
 - PandaCPU 252 LE
 - RAM 176 LE
 - ROM 9 LE
 - Timer 73 LE
 - External Inverter 7 LE
- Fmax 58.5 MHz